Sr.	Experiment Description	Page	Date	Sign
1	Set up Python environment for AI and implement a simple reflex or goal-based agent in a toy environment.			
2	Implement breadth-first search, depth-first search and uniform-cost search on graph or maze problems; compare search cost and path quality.			
3	Implement A* or heuristic search for 8-puzzle/path planning and evaluate the effect of different heuristics.			
4	Develop a game-playing agent using minimax and alpha-beta pruning for Tic-Tac-Toe or a similar turn-based game.			
5	Build a small rule-based expert system or logic inference engine for a diagnosis or advisory problem.			
6	Solve a constraint satisfaction problem such as map coloring, Sudoku or N-Queens using backtracking and consistency checking.			
7	Model a simple Bayesian network or probabilistic classifier for prediction or diagnosis and perform inference using a suitable Python library.			
8	Develop a mini AI application such as text classification, chatbot prototype or perception demo and report on ethics and explainability considerations.			
9	Implement a simple planning problem using STRIPS / goal-stack planning for Blocks World, logistics, or robot navigation.			
10	Implement value iteration or policy-based decision making in a grid-world or similar sequential decision-making environment.			
11	Represent a real-world problem using propositional logic or firstorder logic and perform logical inference to derive conclusions using an AI reasoning approach			
12	Implement a simple AI learning model such as k-NN, decision tree, Naive Bayes, or perceptron and compare results on a toy dataset			

PRACTICAL 1

Set up Python environment for AI and implement a simple reflex or goal-based agent in a toy environment.

```
room = ["D", "C","D","D","C"]
def show_room(room):
    picture = ""
    for spot in room:
      if spot == "D":
        picture += "💩"
      else:
        picture += "✨"
    print(picture)

print("Our Room right now:")
show_room(room)
```

```
Our Room right now:
💩✨💩💩✨
```

```
def clean_spot(spot):
    if spot == "D":
        return "✨"
    else:
        return "✨"

result = clean_spot("d")
print("The robot looked at dirty spot 💩 and made it :", result, "(C means clean ✨)")
```

```
The robot looked at dirty spot 💩 and made it : ✨ (C means clean ✨)
```

```
print("Before the dirty room:")
show_room(room)
print()

for i in range (len(room)):
    room[i] = clean_spot(room[i])
    print("After cleaning the room:", i+1,":")
    show_room(room)

print()
print("After - all done! ✨👼✨")
```

```
Before the dirty room:
✨✨✨✨✨

After cleaning the room: 1 :
✨✨✨✨✨
After cleaning the room: 2 :
✨✨✨✨✨
After cleaning the room: 3 :
✨✨✨✨✨
After cleaning the room: 4 :
✨✨✨✨✨
After cleaning the room: 5 :
✨✨✨✨✨

After - all done! ✨👼✨
```

PRACTICAL 2

Implement breadth-first search, depth-first search and uniform-cost search on graph or maze problems; compare search cost and path quality.

```
map_rooms = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D'],
    'D': ['G'],
    'G': []
}

print("Our Map", map_rooms)
```

```
Our Map {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': ['G'], 'G': []}
```

```
from typing import OrderedDict
from typing_extensions import Doc

def bfs (start, goal):
    to_do=[start]
    visited = []
    order = []
    while to_do:
        room = to_do.pop(0)
        if room in visited:
            continue
        visited.append(room)
        order.append(room)

        if room == goal:
            return order

        for nxt in map_rooms[room]:
            to_do.append(nxt)
    return order

path_order = bfs('A', 'G')
print(path_order)
print(len(path_order))
```

```
['A', 'B', 'C', 'D', 'G']
5
```

```
from typing import OrderedDict
from typing_extensions import Doc

def dfs (start, goal):
    to_do=[start]
    visited = []
    order = []
    while to_do:
        room = to_do.pop()
        if room in visited:
            continue
        visited.append(room)
        order.append(room)

        if room == goal:
            return order

        for nxt in map_rooms[room]:
            to_do.append(nxt)
    return order

path_order = dfs('A', 'G')
print(path_order)
print(len(path_order))
```

```
['A', 'C', 'D', 'G']
4
```

```
door_cost = {
    ('A', 'B'): 1,
    ('B', 'D'): 1,
    ('A', 'C'): 5,
    ('C', 'D'): 1,
    ('D', 'G'): 1
}

def path_cost(path):
    total = 0
    for u in range(len(path) - 1):
       door = (path[u], path[u + 1])
       total += door_cost[door]
    return total

path1 = ['A', 'B', 'D', 'G']
path2 = ['A', 'C', 'D', 'G']
print(path_cost(path1))
print(path_cost(path2))
```

```
3
7
```

```
cost1 = path_cost(path1)
cost2 = path_cost(path2)

print(f"Cost of Path 1: {cost1}")
print(f"Cost of Path 2: {cost2}")

if cost1 < cost2:
    print("Path 1 is better (lower cost).")
else:
    print("Path 2 is better (lower cost).")
```

```
Cost of Path 1: 3
Cost of Path 2: 7
Path 1 is better (lower cost).
```

PRACTICAL 3

Implement A* or heuristic search for 8-puzzle/path planning and evaluate the effect of different heuristics.

```
def astar(start, goal, get_neighbours, guess):
    # The to-do list. Each note is [f, g, place, path_so_far].
    frontier = [[guess(start), 0, start, [start]]] #creating to do list
    visited = []            # places we already explored
    explored_count = 0      # how many places we explored (for comparing)

    while len(frontier) > 0: #keep searching until the list is empty
        # 1) Find the note with the SMALLEST f by looking through the list.
        best = 0 #assume the first element in the list is the best
        for i in range(len(frontier)):
            if frontier[i][0] < frontier[best][0]: #does our frontier has the less f value than our current best one
                best = i
        note = frontier.pop(best)     # take that note out of the list

        f, g, place, path = note      # unpack the note

        # 2) If we already explored this place, skip it.
        if place in visited:
            continue
        visited.append(place)
        explored_count = explored_count + 1

        # 3) Did we reach the goal? Then we are done.
        if place == goal:
            return path, explored_count

        # 4) Add each neighbour to the to-do list as a new note.
        for nxt in get_neighbours(place):
            if nxt not in visited: #only consider the neighbour if we have not already explored it
                new_g = g + 1                       # one more step #every moment has a cost 1
                new_f = new_g + guess(nxt)          # f = g + h
                frontier.append([new_f, new_g, nxt, path + [nxt]]) #take the old path and add the new place to it

    return None, explored_count     # no path found

print("A* is ready. ✅")
```

```
A* is ready. ✅
```

```
grid = [
    "...........",
    "....###....",
    "S.........G",
    "....###....",
    "...........",
]

def find(letter):
    for r in range(len(grid)):
        for c in range(len(grid[r])):
            if grid[r][c] == letter:
                return (r, c)

start = find("S")
goal  = find("G")
rows, cols = len(grid), len(grid[0])

print("Start S is at", start)
print("Goal  G is at", goal)
print()
for line in grid:
    print("  " + "  ".join(line))
```

```
Start S is at (2, 0)
Goal  G is at (2, 10)

  .  .  .  .  .  .  .  .  .  .  .
  .  .  .  .  #  #  #  .  .  .  .
  S  .  .  .  .  .  .  .  .  .  G
  .  .  .  .  #  #  #  .  .  .  .
  .  .  .  .  .  .  .  .  .  .  .
```

```
def grid_neighbours(box):
    r, c = box
    out = []
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:   # up, down, left, right
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:           # stay on the grid
            if grid[nr][nc] != "#":                     # not a wall
                out.append((nr, nc))
    return out


print("From the start", start, "the robot can step to:", grid_neighbours(start))
```

```
From the start (2, 0) the robot can step to: [(1, 0), (3, 0), (2, 1)]
```

```
# Guess 1: no hint at all
def grid_zero(box):
    return 0

# Guess 2: the Manhattan distance (rows apart + columns apart)
def grid_manhattan(box):
    r, c = box
    return abs(r - goal[0]) + abs(c - goal[1])

print("Manhattan guess from the start:", grid_manhattan(start))
```

```
Manhattan guess from the start: 10
```

```
path_zero, explored_zero = astar(start, goal, grid_neighbours, grid_zero)
path_manh, explored_manh = astar(start, goal, grid_neighbours, grid_manhattan)

print("Guess          Path length    Boxes explored")
print("-" * 46)
print("Zero (no hint) ", len(path_zero) - 1, "steps      ", explored_zero)
print("Manhattan      ", len(path_manh) - 1, "steps      ", explored_manh)
print()
print("Same shortest path. But Manhattan explores FEWER boxes. ⭐")
```

```
Guess          Path length    Boxes explored
----------------------------------------------
Zero (no hint)  10 steps       43
Manhattan       10 steps       11

Same shortest path. But Manhattan explores FEWER boxes. ⭐
```

```
def show_grid_path(path):
    path_set = set(path)
    for r in range(rows):
        line = ""
        for c in range(cols):
            if (r, c) == start:
                line += " S "
            elif (r, c) == goal:
                line += " G "
            elif (r, c) in path_set:
                line += " * "
            else:
                line += " " + grid[r][c] + " "
        print(line)

print("The robot's path:")
print()
show_grid_path(path_manh)
```

```
The robot's path:

 .  .  .  .  .  .  .  .  .  .
 .  .  .  .  #  #  #  .  .  .
 S  *  *  *  *  *  *  *  *  G
 .  .  .  .  #  #  #  .  .  .
 .  .  .  .  .  .  .  .  .  .
```

```
goal_state  = "123456780"     # the tidy goal (0 is the empty space)
start_state = "123450678"     # a scrambled start

def show_puzzle(state):
    for r in range(3):
        row = state[r*3 : r*3 + 3]
        row = row.replace("0", "_")          # show the empty space as _
        print("   " + "  ".join(row))

print("Start:")
show_puzzle(start_state)
print()
print("Goal:")
show_puzzle(goal_state)
```

```
Start:
   1  2  3
   4  5  _
   6  7  8

Goal:
   1  2  3
   4  5  6
   7  8  _
```

```
def puzzle_neighbours(state):
    out = []
    zero = state.index("0")          # where the empty space is
    r, c = zero // 3, zero % 3
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:          # stay inside the box
            new_zero = nr * 3 + nc
            tiles = list(state)
            # swap the empty space with the tile next to it
            tiles[zero], tiles[new_zero] = tiles[new_zero], tiles[zero]
            out.append("".join(tiles))
    return out

print("From the start, we can reach these arrangements:")
for s in puzzle_neighbours(start_state):
    print("  ", s)
```

```
From the start, we can reach these arrangements:
   120453678
   123458670
   123405678
```

```python
# Guess 1: no hint
def puzzle_zero(state):
    return 0

# Guess 2: count tiles that are NOT in the right place (ignore the empty space)
def puzzle_wrong_tiles(state):
    count = 0
    for i in range(9):
        if state[i] != "0" and state[i] != goal_state[i]:
            count = count + 1
    return count

print("Wrong tiles in the start:", puzzle_wrong_tiles(start_state))
```

```
Wrong tiles in the start: 3
```

```python
ans_zero,  count_zero  = astar(start_state, goal_state, puzzle_neighbours, puzzle_zero)
ans_smart, count_smart = astar(start_state, goal_state, puzzle_neighbours, puzzle_wrong_tiles)

print("Guess            Moves to solve    Arrangements explored")
print("-" * 56)
print("Zero (no hint)  ", len(ans_zero) - 1,  "moves          ", count_zero)
print("Wrong-tiles     ", len(ans_smart) - 1, "moves          ", count_smart)
print()
print("Same number of moves. But the smart guess explores FAR fewer arrangements! ⭐")
```

```
Guess            Moves to solve    Arrangements explored
--------------------------------------------------------
Zero (no hint)   13 moves          2710
Wrong-tiles      13 moves          190

Same number of moves. But the smart guess explores FAR fewer arrangements! ⭐
```

```python
print("Solving the puzzle, one slide at a time:")
print()
for step, state in enumerate(ans_smart):
    print("Move", step, ":")
    show_puzzle(state)
    print()
```

```
   4  5  8
   6  _  7

Move 3 :
   1  2  3
   4  5  8
   _  6  7

Move 4 :
   1  2  3
   _  5  8
   4  6  7

Move 5 :
   1  2  3
   5  _  8
   4  6  7

Move 6 :
   1  2  3
   5  6  8
   4  _  7

Move 7 :
   1  2  3
   5  6  8
   4  7  _

Move 8 :
   1  2  3
   5  6  _
   4  7  8

Move 9 :
   1  2  3
   5  _  6
   4  7  8

Move 10 :
   1  2  3
   _  5  6
   4  7  8

Move 11 :
   1  2  3
   4  5  6
   _  7  8

Move 12 :
   1  2  3
   4  5  6
   7  _  8

Move 13 :
   1  2  3
   4  5  6
   7  8  _
```

PRACTICAL 4

Develop a game-playing agent using minimax and alpha-beta pruning for Tic-Tac-Toe or a similar turn-based game.

```
board = [' '] * 9

print(board)
```

```
[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
```

```
def show_board(board):
    print()
    print(board[0], '|', board[1], '|', board[2])
    print('--+---+--')
    print(board[3], '|', board[4], '|', board[5])
    print('--+---+--')
    print(board[6], '|', board[7], '|', board[8])
    print()

show_board(board)
```

```
  |   |
--+---+--
  |   |
--+---+--
  |   |
```

```
WINNING_LINES = [
    [0, 1, 2], [3, 4, 5], [6, 7, 8],
    [0, 3, 6], [1, 4, 7], [2, 5, 8],
    [0, 4, 8], [2, 4, 6]
]
```

```
def winner(board):
    for a, b, c in WINNING_LINES:
        if board[a] != ' ' and board[a] == board[b] == board[c]:
            return board[a]

    return None
```

```
def empty_boxes(board):
    return [i for i in range(9) if board[i] == ' ']
```

```
def minimax(board, is_max_turn, alpha, beta):
    # Check whether the game is already finished
    win = winner(board)

    if win == 'X':
        return 1

    if win == 'O':
        return -1

    if len(empty_boxes(board)) == 0:
        return 0

    # MAX turn: AI chooses the highest score
    if is_max_turn:
        best_score = -2

        for box in empty_boxes(board):
            board[box] = 'X'
            score = minimax(board, False, alpha, beta)
            board[box] = ' '

            best_score = max(best_score, score)
            alpha = max(alpha, best_score)

            if beta <= alpha:
                break

        return best_score

    # MIN turn: opponent chooses the lowest score
    else:
        best_score = 2

        for box in empty_boxes(board):
            board[box] = 'O'
            score = minimax(board, True, alpha, beta)
            board[box] = ' '

            best_score = min(best_score, score)
            beta = min(beta, best_score)

            if beta <= alpha:
                break

        return best_score
```

```
def best_move(board):
    best_score = -2
    best_box = None

    for box in empty_boxes(board):
        board[box] = 'X'
        score = minimax(board, False, -2, 2)
        board[box] = ' '

        if score > best_score:
            best_score = score
            best_box = box

    return best_box
```

```
test_board = [
    'X', 'O', 'X',
    'O', 'X', ' ',
    ' ', ' ', 'O'
]

show_board(test_board)

move = best_move(test_board)

print('AI chooses position:', move)
```

```
X | O | X
--+---+--
O | X |
--+---+--
  |   | O

AI chooses position: 6
```

```
def play_game():
    board = [' '] * 9

    print('You are O. The AI is X.')
    print('Choose positions using numbers 0 to 8.')

    while True:
        # AI move
        ai_move = best_move(board)
        board[ai_move] = 'X'

        print('\nAI chooses:', ai_move)
        show_board(board)

        if winner(board) == 'X':
            print('AI wins!')
            break

        if len(empty_boxes(board)) == 0:
            print('Draw!')
            break

        # Human move
        while True:
            try:
                player_move = int(input('Your move (0-8): '))

                if player_move in empty_boxes(board):
                    break

                print('That position is not available.')

            except ValueError:
                print('Please enter a number from 0 to 8.')

        board[player_move] = 'O'
        show_board(board)

        if winner(board) == 'O':
            print('You win!')
            break

        if len(empty_boxes(board)) == 0:
            print('Draw!')
            break
```

PRACTICAL 5

Build a small rule-based expert system or logic inference engine for a diagnosis or advisory problem.

```
def laptop_expert(turns_on, internet_works, is_slow):
    if turns_on == "no":
        return "Check the charger and power cable."

    elif internet_works == "no":
        return "Check Wi-Fi and restart the router."

    elif is_slow == "yes":
        return "Close unused programs and restart the laptop."

    else:
        return "No common problem was found by this expert system."
```

```
turns_on = input("Does the laptop turn on? yes/no: ").lower()
internet_works = input("Does the internet work? yes/no: ").lower()
is_slow = input("Is the laptop slow? yes/no: ").lower()

advice = laptop_expert(turns_on, internet_works, is_slow)

print("\nExpert System Advice:")
print(advice)
```

```
Does the laptop turn on? yes/no: yes
Does the internet work? yes/no: yes
Is the laptop slow? yes/no: yes

Expert System Advice:
Close unused programs and restart the laptop.
```

PRACTICAL 6

Solve a constraint satisfaction problem such as map coloring, Sudoku or N-Queens using backtracking and consistency checking

```
map_data = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']
```

```
def is_consistent(region, color, assignment):
    for neighbor in map_data[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False

    return True
```

```
def solve_map(assignment):
    # If every region has a color, we are done
    if len(assignment) == len(map_data):
        return assignment

    # Choose the next uncolored region
    for region in map_data:
        if region not in assignment:
            break

    # Try each color
    for color in colors:
        if is_consistent(region, color, assignment):
            assignment[region] = color

            result = solve_map(assignment)

            if result is not None:
                return result

            # Backtrack
            del assignment[region]

    return None
```

```
solution = solve_map({})

print("Solution:")

for region, color in solution.items():
    print(region, "=", color)
```

```
Solution:
A = Red
B = Green
C = Blue
D = Red
```